Episode 6: Automated Testing for Network Changes

Video Title

"Network as Code: 31 Tests That Prove Your Network Actually Works"

Target Length

18-22 minutes

Goal

Show the difference between "configured correctly" and "working correctly." Run the post-change test suite, show what each category tests, and demonstrate a test catching a real issue. Make it clear that deploying without testing is like shipping code without running tests.

Pre-Recording Checklist

Before you hit record, make sure:
- ContainerLab topology is running with configs deployed (OSPF/BGP converged)
- All post-change tests currently passing
- Know how to intentionally break something to trigger a test failure (e.g., shut down an interface, kill a BGP session)
- Terminal ready with pytest commands

SEGMENT 1: Recap and the Problem (0:00 - 2:30)

What to show: Quick deployment from last episode, then the question.

Talking points:
- "Last episode, we deployed configs through a CI/CD pipeline. The pipeline validated the data model, generated configs, and pushed them to devices. But there is one question the pipeline did not answer: did it work?"
- "Deployed successfully and working correctly are two very different things. A config can apply cleanly and still break the network. A BGP neighbor statement can be configured perfectly but the session stays in Active state because of a firewall rule or a route policy."
- "Lab 5 adds 31 automated tests that verify the network after every deployment. Not just 'did the config load' but 'are the OSPF neighbors in Full state, are the BGP sessions established, can devices reach each other through the overlay.'"

SEGMENT 2: The Three Test Categories (2:30 - 5:00)

What to show: Simple slide or just talking to camera.

Talking points:
- "The tests are organized into three categories, and each one catches a different kind of problem."
- "Category one: configuration verification. Did the config on the device match what we intended? This compares the running config against the data model. If the OSPF area is supposed to be 0 and it is area 1, this catches it."
- "Category two: operational state. Is the network actually converged? Are OSPF adjacencies Full? Are BGP sessions Established? Are the right routes in the routing table? This is the difference between 'configured' and 'working.'"
- "Category three: health checks. Is the network healthy? Interface error counters, CPU and memory utilization, critical log messages. Your change might have worked perfectly, but if it pushed spine1's CPU to 90%, you need to know."

SEGMENT 3: Running the Full Suite (5:00 - 8:00)

What to show: Run the full test suite.

```
uv run pytest tests/post_change/ -v
```

Talking points:
- "Let me run the full suite against the live fabric."
- Let the tests run. Show the output scrolling.
- "31 tests, all passing. Let me walk through what each section just verified."
- Point out the test names in the output: "testospfneighborsfull, testbgpsessionsestablished, testroutecount, testpingmesh. Each test name tells you exactly what it checked."

SEGMENT 4: Deep Dive - Operational State Tests (8:00 - 12:00)

What to show: Run just the operational tests with verbose output.

```
uv run pytest tests/post_change/test_operational.py -v
```

Talking points:

OSPF Tests

- "The OSPF tests connect to every device via Scrapli, run 'show ip ospf neighbor,' and verify that every expected adjacency is in Full state. Not Init, not 2-Way, not ExStart. Full."
- "If OSPF is stuck in ExStart, something is wrong with MTU or authentication. The test catches it immediately."

BGP Tests

- "The BGP tests verify that every iBGP session is Established. They check the address families. They

verify the route reflector relationships match what the data model declares."

- "A BGP session in Active state means the TCP connection failed. A session in OpenConfirm means the parameters did not match. The tests distinguish between these."

Route Verification

- "Route tests check that each device has the expected number of routes. If spine1 should have 16 routes and it has 12, something did not converge."

Ping Mesh

- "The ping mesh tests verify end-to-end reachability. Every device pings every other device through the overlay. This is the ultimate test: can traffic actually flow?"

SEGMENT 5: Breaking Something on Purpose (12:00 - 16:00)

What to show: Intentionally break the network and watch the tests catch it.

Step 1: Break something

SSH into a device and shut down a fabric interface:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "shutdown"
```

- "I just shut down spine1's interface to leaf1. This is an out-of-band change. Someone SSHed in and broke something. The data model still says this link should be up."

Step 2: Run the tests

```
uv run pytest tests/post_change/test_operational.py -v
```

Talking points:
- "Watch the failures roll in."
- "testospfneighborsfull: FAILED. spine1 lost its adjacency with leaf1. And leaf1 lost its adjacency with spine1."
- "testbgpsessionsestablished: might still pass, because leaf1 still has its BGP session through spine2. But the OSPF test caught the link failure."
- "testpingmesh: depending on the topology, this might pass because traffic rerouted through spine2. ECMP means the network is resilient. But the OSPF test still flagged the degraded state."
- "This is exactly the kind of thing that goes unnoticed in production. The network is still working because of redundancy, but it is running degraded. One more failure and you have an outage. The tests caught it."

Step 3: Fix it

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "no shutdown"
```

Wait a moment for OSPF to reconverge, then rerun:

```
uv run pytest tests/post_change/test_operational.py -v
```

- "All passing again. The tests confirmed the fix."

SEGMENT 6: Health Checks (16:00 - 18:00)

What to show: Run the health tests.

```
uv run pytest tests/post_change/test_health.py -v
```

Talking points:
- "Health tests are the silent watchers. They do not check configuration or protocol state. They check whether the network is stressed."
- "Interface error counters: are any fabric links accumulating CRC errors or drops? Even if OSPF is Full, CRC errors mean you have a physical layer problem building up."
- "Resource utilization: is CPU or memory spiking on any device? A route leak can cause a BGP table explosion that eats memory before anyone notices."
- "These tests run after every deployment, but they are also useful as a standalone health check. Run them anytime you want a quick sanity check on the fabric."

SEGMENT 7: Pipeline Integration (18:00 - 19:30)

What to show: Reference back to the CI/CD pipeline from Episode 5.

Talking points:
- "In the pipeline from Lab 4, these tests run automatically after every deployment. The deploy workflow pushes configs, then immediately runs the post-change test suite."
- "If any test fails, the pipeline reports it. You know within minutes whether your change broke something. Not hours later when a user opens a ticket."
- "This is the full loop: validate the intent, generate configs, deploy, test. Every step automated, every step with a gate. The network never enters a state that the tests have not verified."

SEGMENT 8: The Close (19:30 - 21:00)

What to show: Series overview or back to camera.

Talking points:
- "That is Lab 5. 31 automated tests across three categories: configuration verification, operational state, and health checks. Together they answer the question 'did the deployment actually work?'"
- "The build guide walks through writing every test, setting up the Scrapli fixtures, and integrating with the CI/CD pipeline. Available at [your link] for $49."
- "Next episode, we tackle the scariest problem in network automation: drift. Someone SSHes into a device and makes a manual change. How do you detect it? How do you fix it? That is Lab 6."
- "Subscribe, and I will see you in the lab."

Commands Reference (Quick Copy)

```
# Full test suite
uv run pytest tests/post_change/ -v

# Individual test categories
uv run pytest tests/post_change/test_config_verify.py -v
uv run pytest tests/post_change/test_operational.py -v
uv run pytest tests/post_change/test_health.py -v

# Break something for demo
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c
"shutdown"

# Fix it
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "no
shutdown"
```

Do NOT Show On Camera

- The pytest test code (paid content)
- The Scrapli fixtures and conftest.py (paid content)
- How to write parametrized network tests (paid content)
- The step-by-step process of building the test suite (paid content)

DO Show On Camera

- Test suite output (pass/fail, test names, error messages)
- The three test categories and what they catch
- A live demo of breaking the network and watching tests fail
- The fix and re-verification
- Test counts and coverage